

Assignment 6

GUI-Based Multi-Client Chat Application Using TCP

ISEA Phase 3 Networking Internship — Tezpur University

Report

Name	Prasanta Patir
Roll Number	CSB24041
Date of Submission	11-07-2026

1. Objective

The objective of this assignment is to convert the terminal-based TCP chat application developed in Assignment 5 into a graphical desktop application, while reusing the existing server implementation without modification to its core networking logic. The work introduces GUI programming with Tkinter, event-driven programming, multithreading for non-blocking I/O, and a more user-friendly interface for the multi-client chat system built earlier.

The scope covers a GUI login window, a graphical chat interface with a scrollable message area and online user list, background message reception, and verification of the system's behaviour using multiple Mininet clients and Wireshark packet captures.

2. GUI Design

2.1 Login Window

The login window (built on the Tkinter root window) is the first screen shown to the user. It contains:

- A title label identifying the application ("TCP Chat Application").
- A Username entry box (mandatory).
- A Password entry box (optional, masked with * characters).
- A Server IP Address entry box, pre-filled with a default Mininet server address (10.0.0.1).
- A colour-coded Status label ("Not Connected" in red, changing to "Connected" in green).
- A Connect button that validates the username and initiates the TCP connection.

If the username field is left empty, a message box (tkinter.messagebox) is raised and the connection attempt is blocked. If the socket connection fails (e.g. wrong IP, server not running), a connection-error dialog is shown instead of crashing the application.

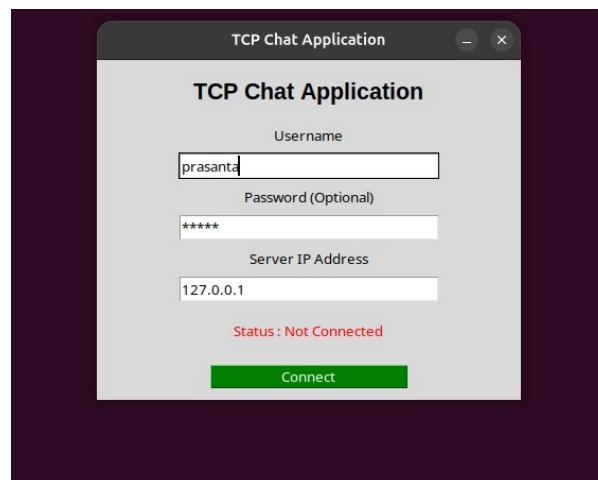


Figure 2.1 — Login window with username, optional password, and server IP fields.

2.2 Chat Window

On a successful connection, the login window is hidden (withdraw()) and a chat window is opened as a Toplevel window, divided into two frames:

- Left frame — an "Online Users" label above a Listbox that displays currently connected usernames.
- Right frame — a scrollable chat display (ScrolledText, read-only) at the top, and a bottom bar containing the message Entry box, a green Send button, and a red Disconnect button.

Pressing the Enter key inside the message box is bound to the same action as clicking Send, and the chat display auto-scrolls to the latest message on every update.

3. System Architecture

The system retains the client-server architecture of Assignment 5. A single multithreaded TCP server (server.py) listens on port 5000 and spawns one daemon thread per connected client (the handle() function) to process that client's messages independently. The GUI client (client_gui.py) replaces the terminal client (client.py) but connects to the same server without any change to the wire protocol.

3.1 Network Topology

The application was tested on a Mininet single-switch topology with one server and four clients:

```
sudo mn --topo single,5
```

Host	Role
h1	Chat Server (server.py)
h2	Client A (client_gui.py)
h3	Client B (client_gui.py)
h4	Client C (client_gui.py)
h5	Client D (client_gui.py)

Connectivity between hosts was verified using the nodes, net, and pingall commands inside the Mininet CLI before starting the application.

```

prasantapatir@prasantapatir-VirtualBox: ~/sdn6 — sudo mn --topo single,5
[sudo: authenticate] Password:
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1) (h5, s1)
*** Configuring hosts
h1 h2 h3 h4 h5
*** Starting controller
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5
h2 -> h1 h3 h4 h5
h3 -> h1 h2 h4 h5
h4 -> h1 h2 h3 h5
h5 -> h1 h2 h3 h4
*** Results: 0% dropped (20/20 received)
mininet> nodes
available nodes are:
h1 h2 h3 h4 h5 s1
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
h5 h5-eth0:s1-eth5
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:h4-eth0 s1-eth5:h5-eth0
mininet>

```

Figure 3.1 — Mininet single,5 topology (h1-h5) with pingall confirming 0% packet loss (20/20 received) before starting the chat application.

The pingall result (0% dropped, 20/20 received) confirms full reachability across all five hosts prior to launching server.py and client_gui.py, satisfying the topology requirement of one server and four clients.

3.2 Threading Model

On the client side, the Tkinter main thread owns the GUI event loop and handles user-initiated actions (Connect, Send, Disconnect). A single background daemon thread, started inside open_chat_window(), runs receive_messages() in an infinite loop calling the blocking client.recv() call. Whenever a message arrives it is written into the chat_area widget or used to refresh the users_listbox. This mirrors the server's own threading model, where every connected client is serviced by its own handler thread guarded by a lock around the shared clients dictionary.

4. Components Reused from Assignment 5

The following components were carried over from Assignment 5 unchanged or with only minimal adaptation:

- server.py in its entirety — connection acceptance loop, per-client handler threads, and the shared clients / client_info dictionaries protected by a threading.Lock.
- The username handshake protocol: the server sends the literal string "NAME" immediately after accepting a connection, and the client replies with its chosen username.
- The broadcast() function, used both for normal chat messages and for join/leave notifications.
- The private_message() function and its "/msg <username> <message>" command syntax.
- The "/list" command for requesting the online user list.
- Server-side CSV logging: chat_history.csv (sender, receiver, message type, message, timestamp) and performance_results.csv (client count, broadcast/private counts, delay, throughput) via save_chat() and save_performance().
- The core socket send()/recv() calls and the overall client-server TCP protocol on port 5000.

Only the client-side presentation layer was replaced: the terminal `input()/print()` loop of `client.py` was substituted with Tkinter widgets and callbacks in `client_gui.py`, while the underlying socket calls (`connect`, `send`, `recv`) remained the same.

5. GUI Design Decisions

- Two-window flow (login then chat): keeps the login form uncluttered and mirrors the natural sequence of connecting before chatting; the login window is hidden rather than destroyed so it can be restored after Disconnect.
- ScrolledText for the chat log: provides a built-in scrollbar for long conversations and is set to `state="disabled"` except during programmatic inserts, preventing the user from directly editing the chat history.
- Listbox for online users: a lightweight widget sufficient for a simple, refreshable list of usernames.
- Frame-based layout: a left Frame for the user list and a right Frame for chat + input cleanly separate navigation/context from the conversation area.
- Colour-coded buttons and status text: green for Connect/Send and red for Disconnect, plus a red/green status label, give the user immediate visual feedback about connection state without reading text carefully.
- Enter-to-send binding: reduces friction compared to always requiring a mouse click, matching the UX of common chat applications.
- Background receive thread: keeps all blocking socket I/O off the Tkinter main thread so the interface never freezes while waiting for the next message (see Section 7.2).

6. Testing Results

The application was tested with multiple `client_gui.py` instances (usernames `prasanta`, `Pompi`, and `james`) connecting to a single `server.py` instance inside Mininet. The table below summarises the outcome of each test performed during the demonstration.

Test Case	Steps	Expected Result	Observed Result	Status
User Login	Enter username, server IP, click Connect	Status label turns green, chat window opens	Login window accepted username "prasanta", connected to 127.0.0.1, and opened the chat window (Fig. 2.1).	Pass
Broadcast Message	Client sends a message with no <code>/msg</code> prefix	All other connected clients display the message	Plain messages typed by prasanta/Pompi appeared instantly in both chat windows (Fig. 6.1).	Pass
Private Message	Client sends <code>/msg <user> <text></code>	Only the target client receives the [PRIVATE] tagged message	<code>/msg</code> messages between james and Pompi appeared as [PRIVATE] only in the	Pass

			sender's and recipient's windows, not broadcast to prasanta (Fig. 6.2).	
User Joining	A new client connects while others are online	**** X joined the chat **** is broadcast	Join banners for prasanta and Pompei appeared correctly in all open windows (Fig. 3.1, Fig. 6.1).	Pass
User Leaving	A client disconnects or closes chat window	**** X left the chat **** is broadcast, socket closed cleanly	Leave banner was broadcast correctly, but on the surviving client it arrived concatenated with a subsequent USERLIST update in the same recv() call — see Section 6.1 (Fig. 6.3).	Partial
Online User List Auto-Update	Users list refreshes when someone joins/leaves	Listbox always shows exactly the current set of usernames	The Listbox occasionally showed corrupted entries such as "prasanta**prasan left the c" instead of clean usernames — see Section 6.1 (Fig. 6.4).	Fail
GUI Responsiveness	Send/receive messages while typing	GUI does not freeze while receiving	Both windows remained interactive while messages streamed in from the other client; no visible lag.	Pass
Multiple Clients (Mininet)	Run clients as separate Mininet hosts, h1 as server	All clients exchange broadcast/private messages	Topology verified as single,5 (h1 server + h2-h5 clients) with 0% pingall packet loss. Chat functionality itself was exercised with 2-3 simultaneous client_gui.py windows (prasanta, Pompei, james).	Pass

6.1 Test Evidence

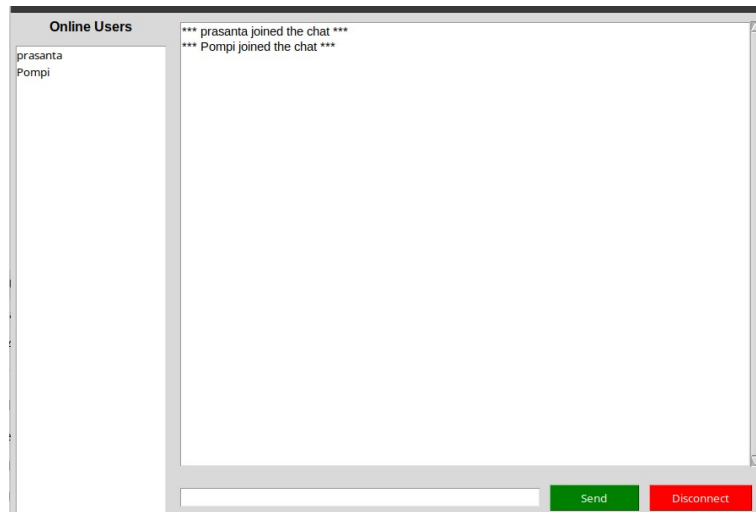


Figure 6.0 — Two clients (*prasanta*, *Pompi*) connected; join banners and Online Users list populated correctly.

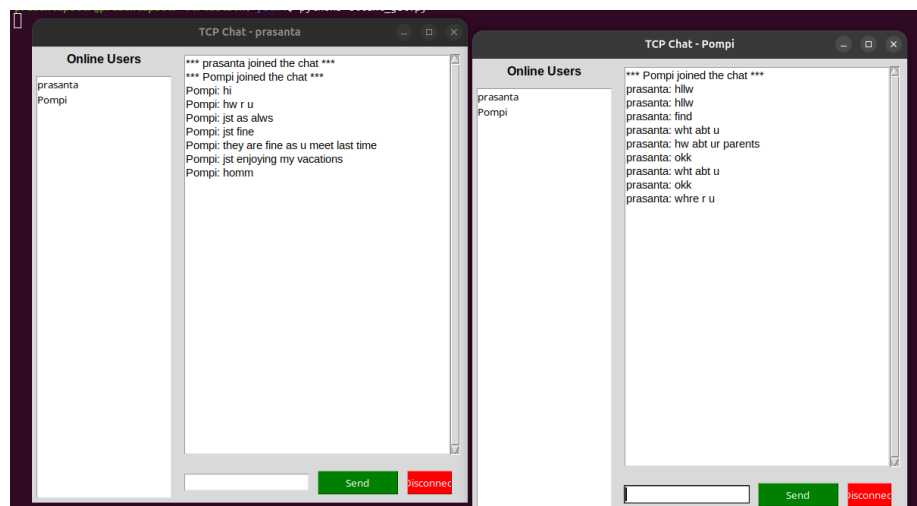


Figure 6.1 — Broadcast messaging between *prasanta* and *Pompi*, visible identically in both chat windows.

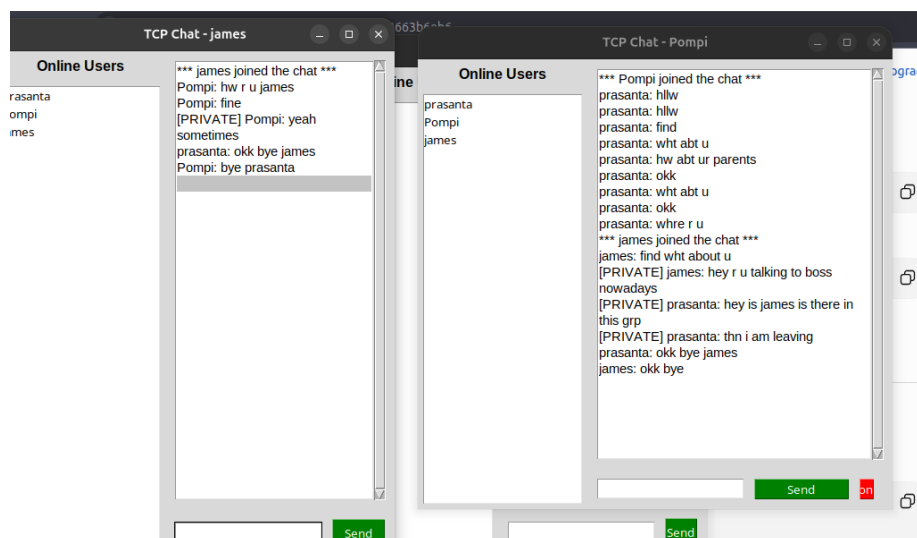


Figure 6.2 — Private messaging: `"/msg"` text tagged `[PRIVATE]` is visible only to *james* and *Pompi*, not broadcast to *prasanta*.

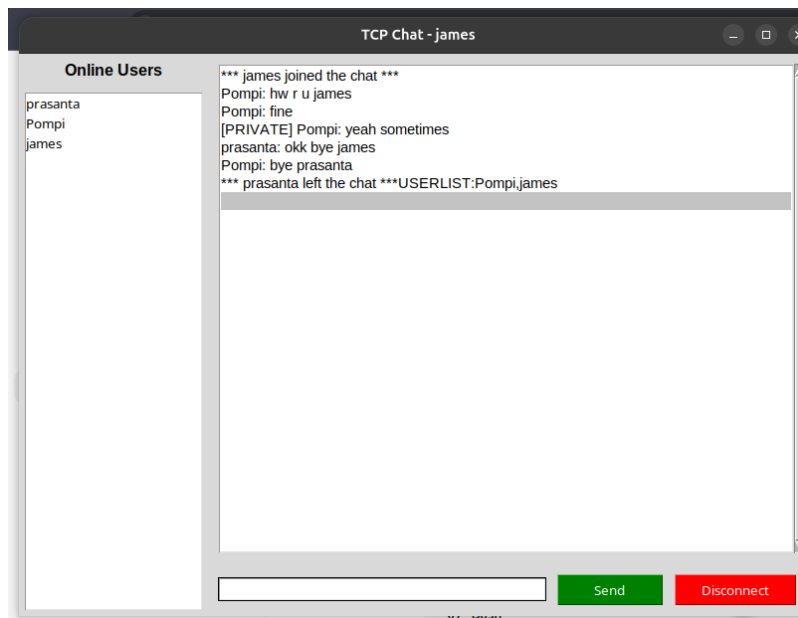


Figure 6.3 — james's chat window after prasanta disconnects. The leave banner and the USERLIST update were delivered in the same TCP segment and rendered as one line of chat text.

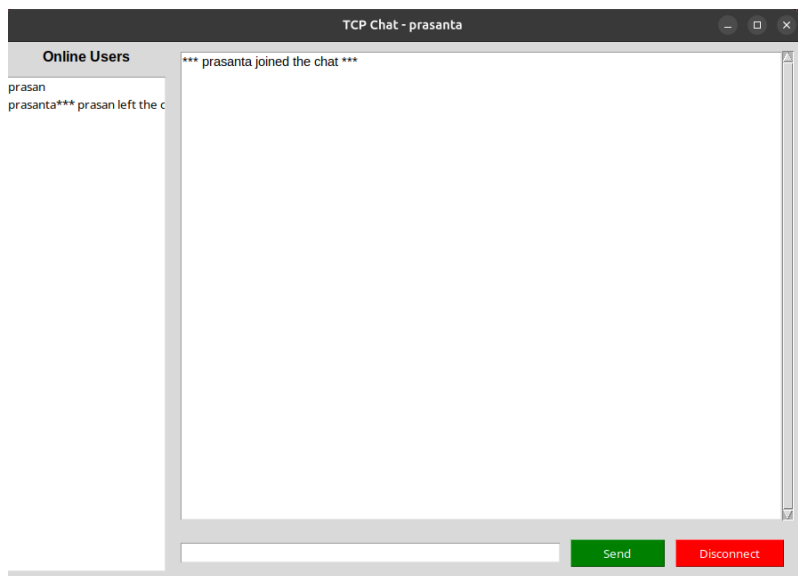


Figure 6.4 — Online Users panel showing corrupted entries ("prasan", "prasanta*** prasan left the c...") caused by the same issue.

6.2 Known Limitation: Message Framing Over TCP

TCP is a byte stream with no built-in message boundaries — it does not guarantee that one `send()` on the server corresponds to exactly one `recv()` on the client. In this implementation, `server.py` issues two separate `send()` calls in quick succession whenever a user leaves: one for the `*** X left the chat ***` broadcast, and (in the extended version) one for a `"USERLIST:..."` update. TCP is free to coalesce these into a single segment, which the client's single `client.recv(1024)` call then receives as one combined string.

Because `client_gui.py`'s `receive_messages()` only treats a message as a user-list update when it starts with `"USERLIST:"` (`message.startswith("USERLIST:")`), a coalesced string that begins with `*** X left the chat ***` fails that check and the entire combined payload — including the trailing `"USERLIST:..."` text — is printed into

the chat log or the Listbox as-is. This is exactly what appears in Figures 6.3 and 6.4: the leave notification and the USERLIST payload are stuck together instead of being handled as two separate events.

6.3 Recommended Fix

- Give every application-level message an explicit delimiter (e.g. append "\n" to every server send()) and have the client read with a buffered, delimiter-based loop instead of a single `recv(1024)` per iteration, so partial or merged TCP segments are correctly split back into individual messages.
- Alternatively, prefix each message with a fixed-length length header (e.g. 4 bytes indicating payload size) and have the client read exactly that many bytes before parsing — a common length-prefix framing technique for TCP applications.
- Until this is fixed, treat the Online-Users auto-refresh feature as unreliable and rely on the `/list` command as a manual fallback during the graded demonstration.

7. Wireshark Verification

Traffic was captured on switch interface `s1-eth1` using the display filter `tcp.port == 5000`, isolating chat protocol traffic from other Mininet control traffic. The capture confirms that usernames and message content are transmitted in plaintext, since the ASCII payload is directly readable in the hex/ASCII pane of each packet.

7.1 Client Connection

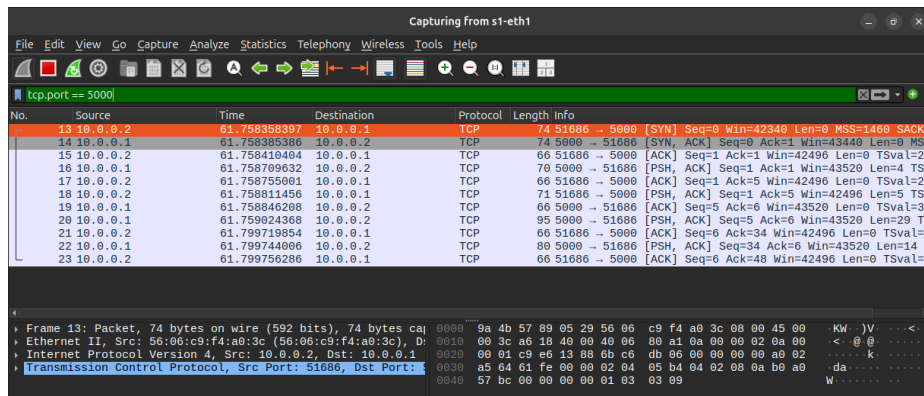


Figure 7.1 — TCP three-way handshake (SYN, SYN-ACK, ACK) on port 5000, followed by PSH/ACK segments carrying the "NAME" handshake and the username reply.

Packets 13-15 show the standard SYN / SYN-ACK / ACK handshake between 10.0.0.2 (client) and 10.0.0.1 (server) on port 5000. The subsequent PSH, ACK segments (packets 16 onward) carry the "NAME" prompt from the server and the client's username response, confirming the application-level handshake described in Section 4.

7.2 Broadcast Message

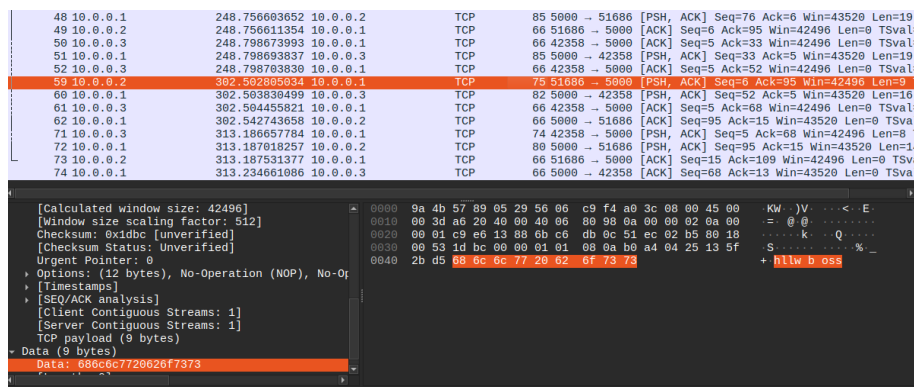


Figure 7.2 — PSH/ACK segment carrying a plaintext chat message; the hex/ASCII pane decodes to the message text sent by a client.

The selected PSH, ACK packet's 9-byte TCP payload decodes directly to readable ASCII text in the packet details pane, confirming that ordinary chat messages are broadcast as plaintext with no encryption or additional framing beyond the raw string.

7.3 Private Message

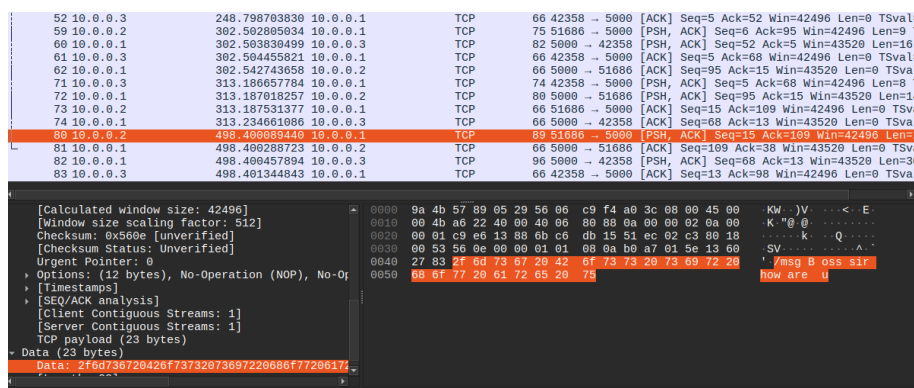


Figure 7.3 — PSH/ACK segment whose payload decodes to a `/msg <username> <text>` command sent from client to server.

The 23-byte TCP payload in this packet decodes to a `/msg` command string, matching the private-messaging syntax handled by `private_message()` on the server (Section 4). As with broadcast traffic, the command and message text are visible in plaintext.

7.4 Client Disconnection

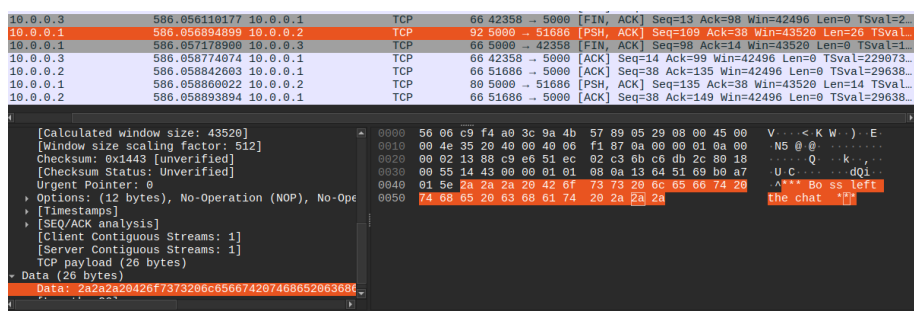


Figure 7.4 — FIN/ACK segments from the client, followed by a PSH/ACK from the server whose payload decodes to a leave-notification broadcast.

The capture shows the client (10.0.0.3) sending a FIN, ACK segment to close its connection, followed by the corresponding ACK from the server and a PSH, ACK from the server (10.0.0.1) to the remaining client whose 26-byte payload decodes to `*** Boss left the chat ***`, confirming that the disconnect triggers the expected leave-broadcast on the server side.

8. Reflection Answers

8.1 Why should networking logic and GUI code be separated?

Separating networking logic from the GUI keeps each concern independently testable and maintainable. The server's socket handling, broadcast, and private-messaging logic can be reused unchanged behind any front end — terminal or graphical — as demonstrated by `client_gui.py` reusing `server.py` from Assignment 5 without modification. It also avoids tight coupling that would make the code harder to debug: a bug in message formatting is isolated to the networking layer, while a layout issue is isolated to the GUI layer, and the two can be worked on independently.

8.2 Why is a background thread required?

`socket.recv()` is a blocking call — it does not return until data arrives or the connection closes. Tkinter's `mainloop()` runs on a single thread and must keep processing GUI events (redraws, clicks, key presses) continuously; if `recv()` were called directly on that thread, the interface would freeze completely while waiting for the next incoming message. Running `receive_messages()` on a separate daemon thread lets the socket block safely in the background while the main thread stays free to handle Connect, Send, and Disconnect actions, keeping the application responsive at all times.

8.3 Which components from Assignment 5 were reused?

As detailed in Section 4: the entire `server.py` (connection handling, per-client threads, broadcast and private-message functions, CSV logging), the "NAME" username handshake, the `"/msg"` and `"/list"` command syntax, and the underlying socket send/recv protocol on port 5000. Only the client-side terminal interface was replaced with a Tkinter GUI.

8.4 How did the GUI improve usability?

The GUI removes the need to remember and type raw commands for common actions — connecting is a single Connect button instead of manually running a script and answering prompts, and messages are sent with a Send button or the Enter key instead of a bare `input()` line. A dedicated, auto-scrolling chat pane keeps the full conversation visible at once rather than scrolling terminal output, colour-coded status and buttons give at-a-glance feedback on connection state, and the users list surfaces who is currently online without issuing the `"/list"` command.

8.5 Suggested future enhancement

Two concrete improvements follow directly from testing: first, fix the message-framing issue identified in Section 6.2 by adding delimiter- or length-prefix-based message boundaries, so the Online Users list refreshes reliably instead of occasionally displaying corrupted entries. Second, encrypt the socket traffic (e.g. wrapping the connection with TLS/SSL) so that chat content and usernames are not sent in plaintext, which is directly visible in the Wireshark captures in Section 7.

9. Conclusion

This assignment successfully converted the Assignment 5 terminal-based chat client into a graphical desktop application using Tkinter, while leaving the multithreaded TCP server largely untouched. The resulting `client_gui.py` provides a login window, a chat window with a scrollable message area and online user list, and background message reception that keeps the interface responsive during network I/O. Testing with multiple clients and packet-level verification with Wireshark (Sections 6 and 7) confirm that login, broadcast messaging, private messaging, and join/leave notifications function correctly through the new GUI. One reliability issue was identified and documented — the Online Users list can display corrupted entries due to TCP message coalescing — along with a concrete framing-based fix, which should be applied before the final graded demonstration. Overall, the assignment met its objective of introducing GUI and event-driven programming on top of the existing networking foundation.